UNIVERSIDAD AUSTRAL

INGENIERÍA
Departamento de Computación

APUNTES

Sistemas Distribuidos

Clase 01 – Pensamiento distribuido

Departamento de Computación

Ingeniería Informática – Comisión A

Pilar – Primer cuatrimestre 2026

Equipo docente

11 de agosto de 2026

1 Propósito de la clase

Esta primera clase instala el marco mental del curso. Su objetivo no es presentar una lista de tecnologías ni una taxonomía superficial, sino cambiar la forma en que el estudiante razona sobre sistemas que operan a través de una red.

La idea central es la siguiente:

> En un sistema distribuido, el observador no dispone de una visión global completa, instantánea y confiable del estado del sistema.

Esta limitación no es un detalle de implementación ni una rareza de ciertos entornos hostiles. Es una consecuencia estructural de tres hechos básicos:

- los componentes se ejecutan en máquinas distintas;
- se comunican mediante intercambio de mensajes;
- la red introduce demora, pérdida, reordenamiento e incertidumbre.

Por eso, a lo largo de toda la materia, evitaremos pensar como si el sistema fuera un único programa local con un estado perfectamente observable. La meta será desarrollar *pensamiento distribuido*: la capacidad de modelar incertidumbre, explicitar fallas, elegir garantías y defender trade-offs.

2 Qué vuelve distribuido a un problema

No todo problema con varias máquinas es, automáticamente, un problema distribuido interesante. Tampoco alcanza con decir que "hay red", "hay escalabilidad" o "hay muchos usuarios".

Una formulación operativa útil es la siguiente:

> Un problema se vuelve distribuido cuando su corrección depende de coordinar componentes que no comparten memoria, no comparten un reloj global confiable y solo pueden intercambiar información a través de mensajes.

Esta definición importa porque desplaza el eje del razonamiento:

- ya no alcanza con pensar en términos de estado local;
- ya no puede asumirse orden global instantáneo;
- ya no puede suponerse que una falta de respuesta identifica una causa única.

2.1 Consecuencia metodológica

Cuando un problema es distribuido, la pregunta correcta deja de ser:

> "¿Cuál es el estado real exacto del sistema?"

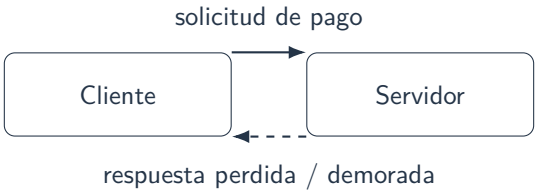
y pasa a ser:

> "¿Qué puede observar este componente, qué no puede saber, y qué garantía se puede ofrecer bajo esa incertidumbre?"

3 Ejemplo motivador: el pago que no responde

Consideremos la siguiente situación:

1. un cliente inicia un pago;
2. el request es enviado al servidor;
3. el servidor procesa la operación;
4. la respuesta no vuelve al cliente.



La pregunta intuitiva es inmediata:

> ¿El pago se realizó o no?

La respuesta correcta, desde el punto de vista del cliente, es:

> No lo sabemos.

Y lo más importante es esto: en general, no podemos saberlo con certeza en ese instante.

4 Indistinguibilidad

La noción fundamental que esta clase introduce es la de **indistinguibilidad observacional**. Distintas historias internas del sistema pueden producir exactamente la misma observación para un actor remoto.

4.1 Dos historias distintas

Volvamos al ejemplo del pago. El cliente observa simplemente que no recibió respuesta en el tiempo esperado. Sin embargo, al menos dos historias muy distintas son compatibles con esa observación:

Historia A El servidor nunca recibió el request. No hubo ningún efecto y el cliente terminó en timeout.

Historia B El servidor sí recibió y procesó el request, pero la respuesta se perdió. Hubo efecto, pero el cliente terminó también en timeout.

4.2 Punto conceptual

Para el cliente, ambas historias son observacionalmente idénticas. En otras palabras:

> La ausencia de respuesta no identifica por sí sola la causa profunda del fenómeno observado.

Esto es central, porque obliga a abandonar una intuición local muy arraigada: la idea de que el sistema "debería poder decirnos exactamente qué pasó".

4.3 Consecuencia fuerte

Desde el borde del sistema, lo que se tiene no es conocimiento perfecto, sino evidencia parcial. Diseñar sistemas distribuidos significa aprender a operar bajo esa limitación.

5 Timeouts no son pruebas

Un error muy frecuente en estudiantes —y también en desarrolladores inexpertos— es asumir que:

> "Si hubo timeout, entonces el servidor falló."

Eso es incorrecto.

Un timeout solo significa que una respuesta no fue observada antes de cierto umbral temporal. Puede deberse a:

- caída del servidor;
- demora grande de red;
- pérdida del request;
- pérdida de la respuesta;
- sobrecarga temporal;
- ejecución lenta;
- combinación de varios factores.

Un timeout es un hecho de observación, no una explicación causal.

6 Modelo síncrono y modelo asíncrono

Para razonar con mayor precisión, conviene distinguir dos modelos.

6.1 Modelo síncrono

En un modelo síncrono se asume la existencia de cotas conocidas para:

- tiempo de procesamiento;
- tiempo de comunicación;
- eventualmente, deriva relativa entre relojes.

En ese contexto, si una operación excede su cota, puede interpretarse razonablemente como una falla.

6.2 Modelo asíncrono

En un modelo asíncrono no existen cotas confiables conocidas para la red o el cómputo. Por lo tanto, una demora arbitrariamente larga es indistinguible de una falla real.

Este modelo es más severo, pero conceptualmente mucho más cercano a la realidad de sistemas desplegados sobre redes generales.

6.3 Importancia para el curso

El curso se apoya principalmente en la intuición asíncrona: si no se puede distinguir claramente entre demora y falla, entonces las soluciones deben diseñarse de forma robusta a esa ambigüedad.

7 Qué significa pensar distributivamente

A partir de lo anterior, podemos definir el núcleo del curso.

> Pensar distributivamente es razonar no desde la ilusión de conocimiento global, sino desde la explicitación de lo que cada actor puede observar, de las fallas que se asumen y de las garantías que se desean preservar.

Esto significa que toda decisión de diseño debe responder, al menos, a estas preguntas:

- ¿Cuál es el problema real?
- ¿Qué fallas estoy modelando?
- ¿Qué garantía quiero preservar?
- ¿Qué solución elijo?
- ¿Qué costo o trade-off acepto?

8 Framework del curso

Durante toda la materia vamos a usar el siguiente esquema:

$$D = (P, F, G, S, T)$$

donde:

- $P =$ problema;
- $F =$ fallas asumidas;
- $G =$ garantía requerida;
- $S =$ solución elegida;
- $T =$ trade-off aceptado.

Este esquema no es decorativo. Su función pedagógica es ordenar el razonamiento y evitar que el estudiante salte directamente a una tecnología o patrón sin haber modelado el problema.

8.1 Ejemplo con el pago

- P: evitar duplicación de pagos;
- F: timeout, pérdida de respuesta, retry del cliente;
- G: a lo sumo un débito efectivo;
- S: request ID único + de-duplicación + persistencia del resultado;
- T: más complejidad, más estado, más lógica de validación.

Lo importante es notar que la solución no aparece en el vacío: surge como respuesta a un problema, a un modelo de fallas y a una garantía requerida.

9 CAP: interpretación correcta

La clase cierra conectando estas ideas con CAP, pero de forma correcta.

CAP no debe entenderse como un slogan del tipo "elegir dos de tres", sino como una restricción que aparece **bajo partición de red**.

Cuando la red se parte, un sistema replicado no puede ofrecer simultáneamente:

- consistencia fuerte observable;
- disponibilidad total de las operaciones.

Eso obliga a decidir entre:

- rechazar ciertas operaciones para preservar corrección observable;
- responder igual para preservar disponibilidad, aceptando posibles inconsistencias.

9.1 Lectura madura

CAP no te dice qué arquitectura usar en abstracto. Te obliga a explicitar qué valorás más cuando la coordinación ya no es posible.

10 Errores conceptuales frecuentes

Esta primera clase busca corregir desde el inicio algunos errores muy comunes:

- creer que timeout implica crash;
- creer que el sistema posee un estado global inmediatamente observable;
- pensar que toda garantía importante puede obtenerse sin costo;
- tratar CAP como slogan, no como restricción;
- asumir que "más tecnología" equivale a "mejor diseño".

11 Qué debe saber hacer el estudiante al terminar esta clase

Al finalizar la clase, el estudiante debería poder:

- explicar qué vuelve distribuido a un problema;
- describir por qué la observación del sistema es incompleta;
- construir dos historias indistinguibles para un cliente;
- aplicar el esquema $D = (P, F, G, S, T)$ a un caso simple;
- explicar CAP como decisión bajo partición.

12 Guía de profundización bibliográfica

- **DDIA / Kleppmann**: leer las discusiones introductorias sobre fallas, visibilidad e imposibilidad de obtener una imagen perfecta del sistema. El valor de esta lectura es conceptual, no solo terminológico.
- **Tanenbaum**: usarlo para reforzar definiciones de sistema distribuido, modelos de sistema y vocabulario básico.
- **MIT 6.824**: releer materiales introductorios pensando especialmente en qué sabe cada actor y qué no puede saber.

Resultado de aprendizaje esperado: al finalizar esta clase, el estudiante debe haber cambiado su modelo mental. Ya no debe razonar como si el sistema fuera una sola máquina local con observación completa. Debe poder razonar desde síntomas hacia causas posibles, documentar el relevamiento y el análisis, y relevar, analizar y proponer en forma aislada y conectada.

13 Síntesis razonada de la clase

Esta unidad debe leerse como parte de una cadena de decisiones de diseño y no como un catálogo de mecanismos. Para estudiar el tema con profundidad conviene reconstruir cuatro preguntas: *qué problema aparece*, *qué garantía se desea*, *qué mecanismo la aproxima* y *qué costo introduce*. En cada ejemplo debe identificarse además el modelo de falla implícito.

Una respuesta técnicamente madura no termina al nombrar un algoritmo o patrón: declara sus supuestos, la propiedad que preserva y el escenario en el que deja de ser suficiente.

14 Bibliografía guiada y ruta de profundización

Para profundizar esta clase se recomienda: Kleppmann, caps. 1–2; Tanenbaum y Van Steen, introducción y modelos.

La lectura debe hacerse con preguntas concretas: (1) ¿qué modelo de sistema asume el autor?, (2) ¿qué propiedad demuestra o persigue?, (3) ¿qué falla o anomalía motiva el mecanismo?, y (4) ¿qué trade-off queda abierto?

15 Conexión con el Trabajo Final Integrador

El grupo debe señalar qué parte de su sistema utiliza los conceptos de esta unidad, justificar por qué son necesarios y documentar una alternativa descartada. Cuando el contenido todavía no corresponda a la etapa vigente, debe registrarse como hipótesis o decisión pendiente.

16 Autoevaluación conceptual

1. ¿Cuál es el problema distribuido concreto que motiva esta clase?
2. ¿Qué propiedad de seguridad y qué propiedad de progreso están en juego?
3. ¿Qué supuesto de red, tiempo o falla resulta decisivo?

4. ¿Qué trade-off aceptaría en un sistema real y cuál no aceptaría en un dominio crítico?
5. ¿Cómo observaría experimentalmente que la solución se comporta como espera?

17 Profundización autocontenida v9

17.1 Problema, límite e invariante

El núcleo de esta clase es incertidumbre, fallas parciales, asincronía, indistinguibilidad y marco P-F-G-S-T. Para estudiarlo de manera autónoma conviene separar tres planos. Primero, el *problema observable*: qué comportamiento incorrecto puede ver un cliente o un operador. Segundo, el *límite del modelo*: qué información, sincronía o coordinación no está disponible. Tercero, el *invariante*: qué propiedad no debe violarse aunque el sistema pierda progreso temporalmente.

El modelo de fallas relevante incluye delay, pérdida, duplicación, partición y observación incompleta. Una solución debe describirse como protocolo y no solo como nombre de tecnología: actores, estados, mensajes, condición de éxito, condición de aborto y estado visible después de una interrupción. La evidencia esperada es timeline de ejecuciones indistinguibles y explicación de qué sabe cada actor.

17.2 Método de análisis

Ante un caso nuevo, siga esta secuencia:

1. Identifique actores, dato o recurso compartido y frontera de confianza.
2. Escriba una ejecución nominal y una ejecución adversarial.
3. Distinga seguridad (lo malo no ocurre) de progreso (algo bueno finalmente ocurre).
4. Declare qué supuesto permite avanzar: mayoría, deadline, sincronía parcial, operación idempotente, almacenamiento durable u otro.
5. Compare una alternativa más simple y una más fuerte; registre qué métrica o experimento haría cambiar la decisión.

17.3 Caso transferible

Considere una plataforma de reservas que recibe operaciones duplicadas, pierde conectividad entre regiones y debe sostener un camino crítico sin ocultar el estado real al usuario. Aplique el mecanismo de esta clase a una única decisión concreta. Una respuesta madura no intenta resolver todo: delimita la operación, formula la garantía y explica la degradación esperada cuando el supuesto central deja de cumplirse.

17.4 Errores de razonamiento frecuentes

- confundir una propiedad con el producto que suele implementarla;
- describir el camino feliz sin recorrer una falla intermedia;
- afirmar “exactly-once”, “alta disponibilidad” o “consistencia” sin definir alcance observable;
- medir solo throughput aunque la hipótesis trate sobre semántica o recuperación;
- presentar la complejidad añadida como beneficio sin compararla con la alternativa más simple.

18 Lectura guiada

Kleppmann, introducción y capítulo 8; Tanenbaum y van Steen, modelos de sistema.

Durante la lectura, registre: modelo de sistema, propiedad perseguida, contraejemplo que motiva el mecanismo, costo principal y una condición bajo la cual no lo elegiría. Estas cinco anotaciones convierten la bibliografía en una herramienta de diseño y no en una lista para memorizar.

19 Aplicación al Trabajo Final Integrador

El equipo debe producir un ADR breve con la decisión asociada a esta clase. Debe contener: contexto, dos alternativas, supuesto decisivo, garantía elegida, trade-off, evidencia pendiente y fecha de revisión. La evidencia mínima esperada para esta unidad es: timeline de ejecuciones indistinguibles y explicación de qué sabe cada actor.

20 Autoevaluación avanzada

1. ¿Puedo explicar la clase sin nombrar primero una tecnología?
2. ¿Puedo construir una ejecución que viole la garantía si el mecanismo se configura mal?
3. ¿Puedo distinguir seguridad, progreso y experiencia del usuario?
4. ¿Puedo proponer una medición que valide la propiedad, no solo el rendimiento?
5. ¿Puedo defender cuándo una solución más simple sería suficiente?